Machine Learning Fundamentals

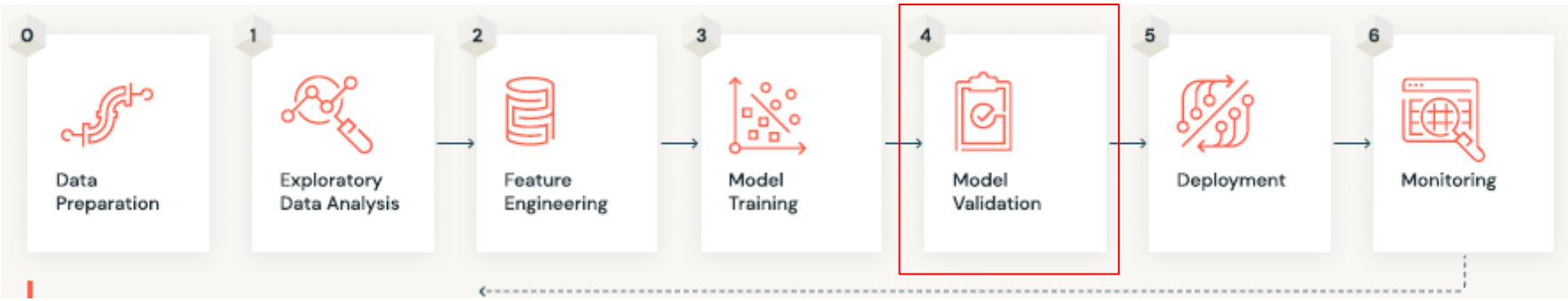
EVALUATION METRICS
FOR BINARY CLASSIFICATION

Andy Louwyck
andy.louwyck@vives.be
Academic year 2025 - 2026

hogeschool vives

Who, what, why, and when?

- Evaluating machine learning models
  - Validating
  - Testing
- By the machine learning engineer
  - Easy to interpret by humans
- To compare models objectively
  - Selecting the most performing model

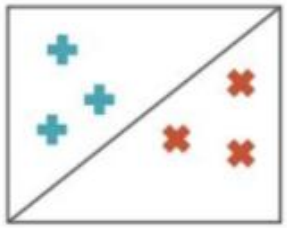


Already seen…

- Linear regression
- Polynomial regression
- Perceptron
- Logistic regression

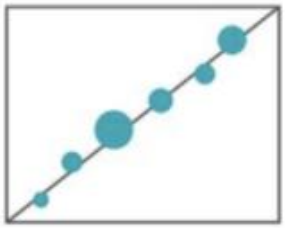
How do we evaluate the performance of ML models?

- Regression: MAE, MSE, RMSE, R^2, …
- Classification: Accuracy?



CLASSIFICATION

Sorting items into categories



REGRESSION

Identifying real values (dollars, weight, etc.)

Two datasets as an example

- Medical dataset:
  - 1000 people: COVID?
    - 10 infected
    - 990 not infected

- Email dataset:
  - 100 Emails: Spam or Ham?
    - 40 spam
    - 60 ham

Accuracy: Definition

- **Definition**: number of correct predictions divided by the total number of samples:

$$acc = \frac{\#\ correct\ predictions}{\#\ samples}$$

- Example:
  - Model is trained with 1000 examples
  - 875 examples were classified correctly
  - Accuracy = 875/1000 = 0.875 = 87.5%

A very accurate test…

Is a patient infected with COVID?

- Always predict negatively
- Accuracy is 99%

```
def predict_covid(patient_data):
    return False
```

We need other evaluation metrics!

Two types of correct predictions

Binary classification: positive and negative labels

- Medical dataset:

  Positive = infected; Negative = not infected

- Email dataset:

  Positive = spam; Negative = ham

True positives (TP):

- Infected patients who test positive
- Unwanted emails that end up in the spam folder

True negatives (TN):

- Healthy patients who test negative
- Desired emails that arrive in the inbox

hogeschool vives

Two types of incorrect predictions

False positives (FP):

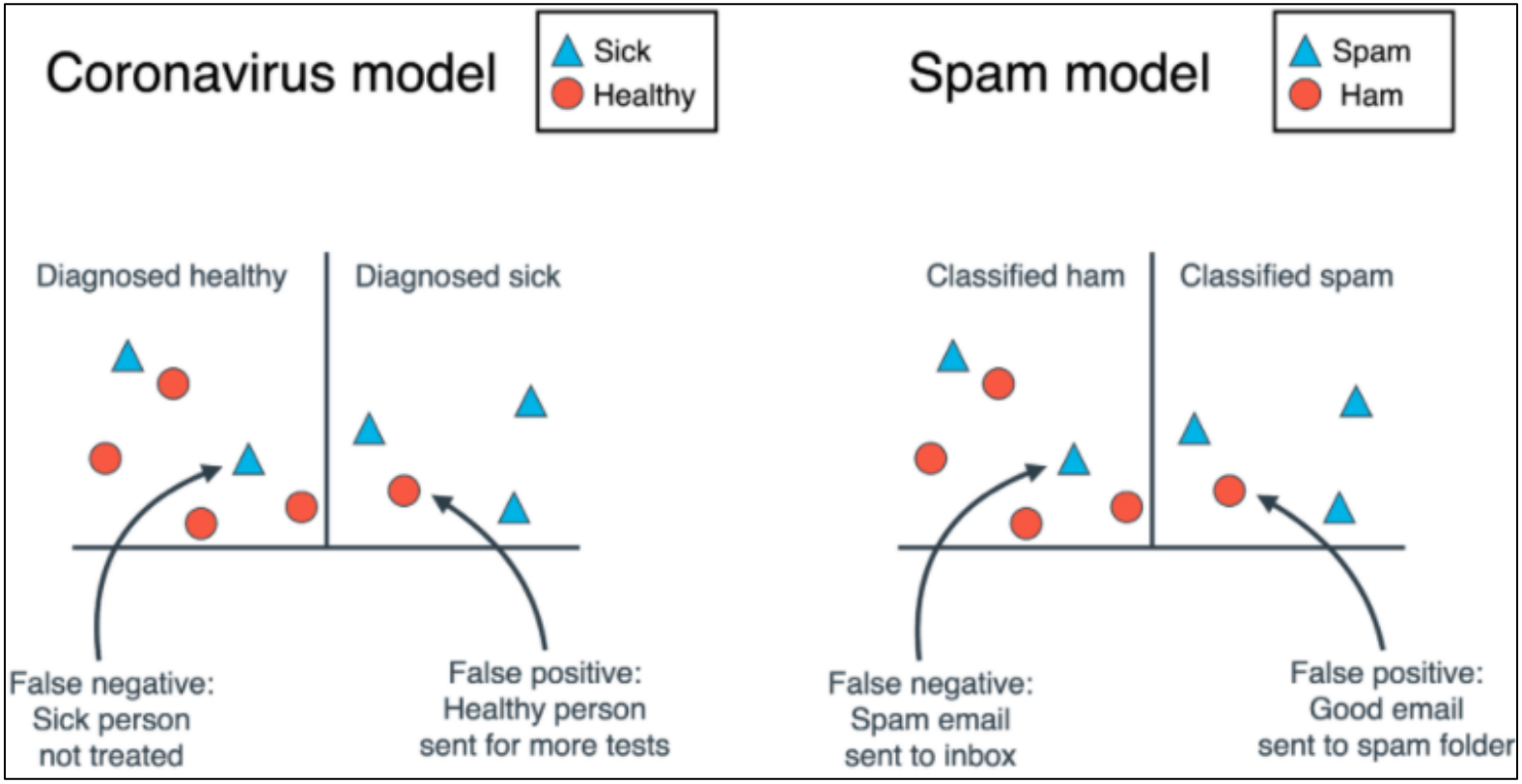
- Healthy patients who test positive
- Desired emails that end up in the spam folder

False negatives (FN):

- Infected patients who test negative
- Unwanted emails ending up in the inbox

hogeschool vives

Example


Coronavirus model
Sick
Healthy
Diagnosed healthy
Diagnosed sick
False negative:
Sick person
not treated
False positive:
Healthy person
sent for more tests
Spam model
Spam
Ham
Classified ham
Classified spam
False negative:
Spam email
sent to inbox
False positive:
Good email
sent to spam folder

Analysis of the 2 datasets

What do we prefer? False positives or false negatives?

- COVID dataset:
  - Better to have a healthy person test positive

  then have the infected person tested negative
  - So we prefer false positives here
- Email dataset:
  - We prefer to receive unwanted mail in our mailbox

  then desired mail in our spam folder
  - So here we prefer false negatives

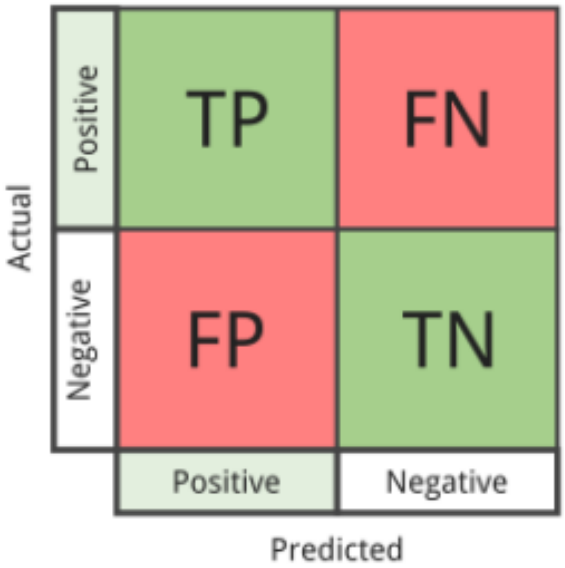
Conclusion: we need metrics other than accuracy!

hogeschool vives

Confusion Matrix

a table used to evaluate the performance of a classification model by showing the counts of

- **True Positives (TP)**
- **True Negatives (TN)**
- **False Positives (FP)**
- **False Negatives (FN)**



Accuracy = $\frac{TP+TN}{TP+TN+FP+FN}$

(Note: the predictions can also be in the rows and the actual values in the columns)

Confusion Matrix: Example

Person's condition	Predicted positives	Predicted negatives	Total
Actual positives	number of true positives (TP)	number of false negatives (FN)	TP + FN
Actual negatives	number of false positives (FP)	number of true negatives (TN)	FP + TN
Total	TP+FP	FN+TN	TP + TN + FP + FN

Example: the simple and accurate corona test:

```
def predict_covid(patient_data):
    return False
```

Coronavirus model 1	Predicted sick	Predicted healthy	Total
Actual sick	0	10	10
Actual healthy	0	990	990
Total	0	1000	1000

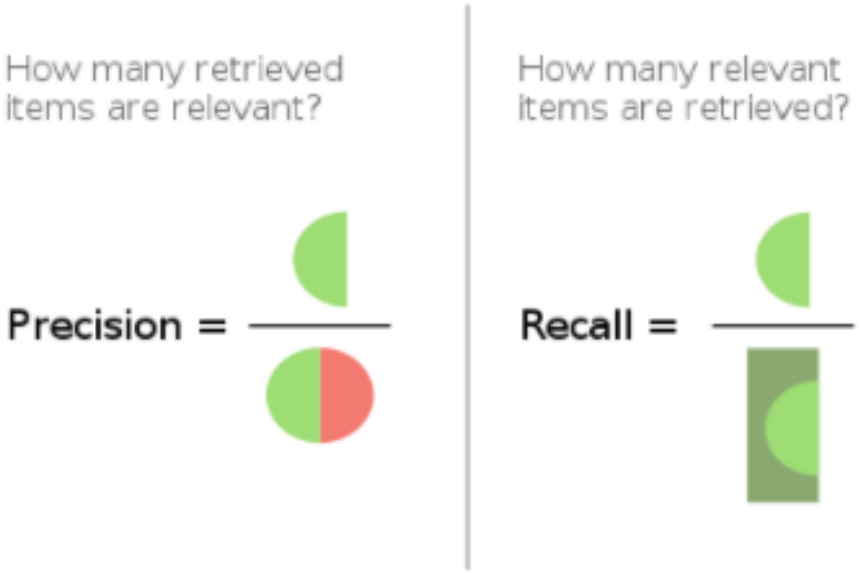
None of the sick people test positive, so the test is worthless!

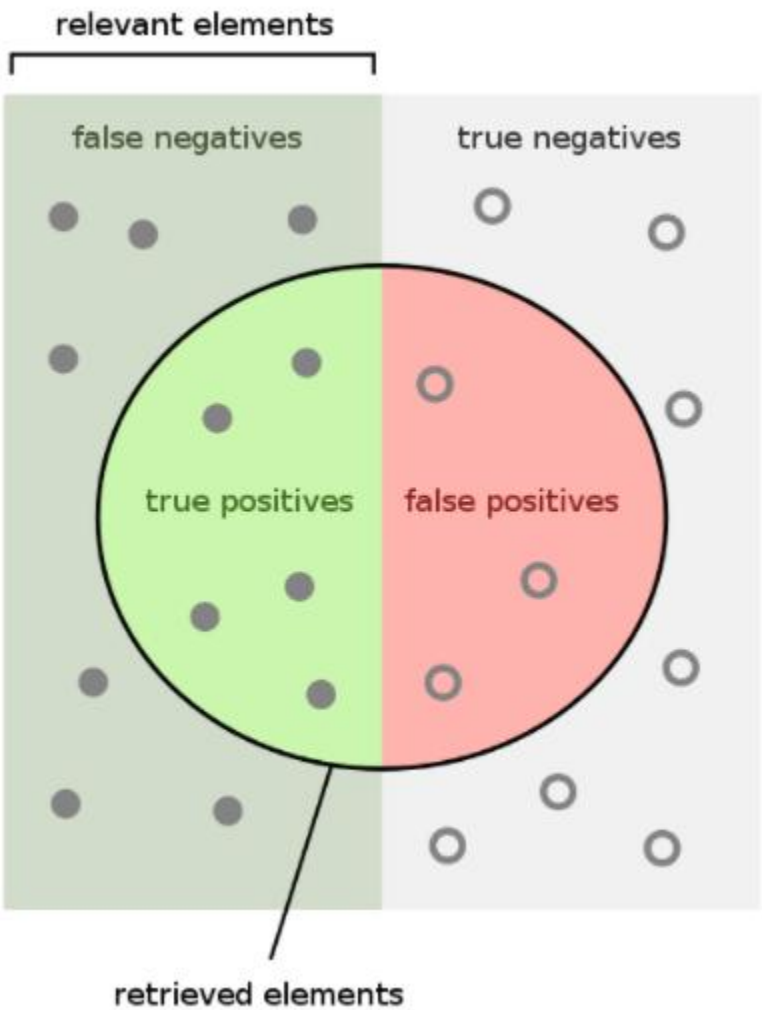
→ The confusion matrix provides a more complete picture

→ From the confusion matrix we can derive new metrics

Recall and Precision: Definitions

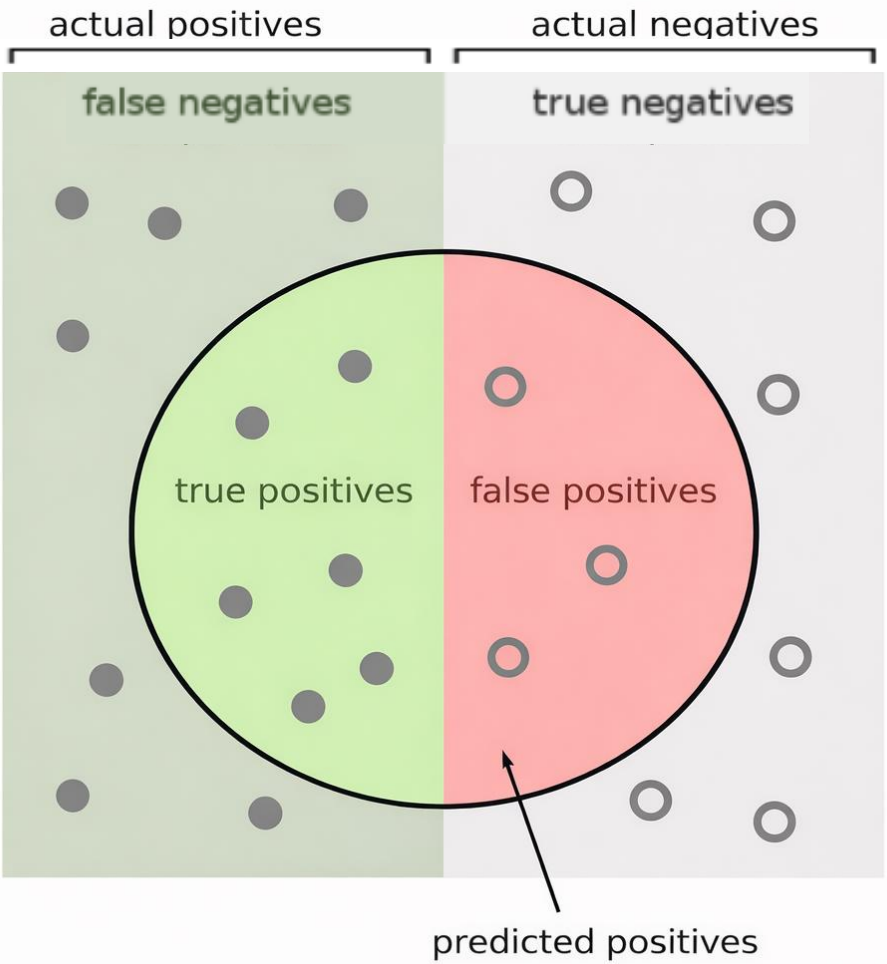
- **Recall =** $\frac{TP}{TP+FN} = \frac{true\ positives}{actual\ positives}$
- **Precision =** $\frac{TP}{TP+FP} = \frac{true\ positives}{predicted\ positives}$





https://en.wikipedia.org/wiki/Precision_and_recall

Recall and Precision: Definitions



How many predicted positives are actually positive?

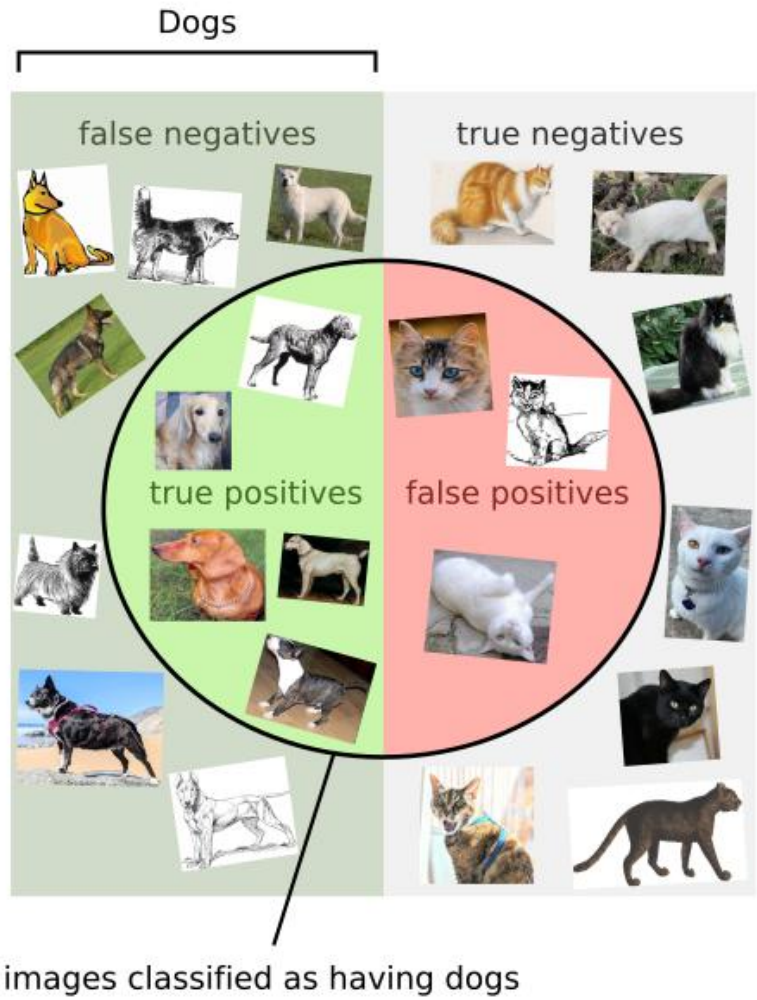
Precision =

How many actual positives are predicted as positive?

Recall =

hogeschool vives

Recall and Precision: Example



$$\text{Precision} = \frac{5 \text{ true pos.}}{8 \text{ total pos.}}$$

$$\text{Recall} = \frac{5 \text{ true pos.}}{12 \text{ total dogs}}$$

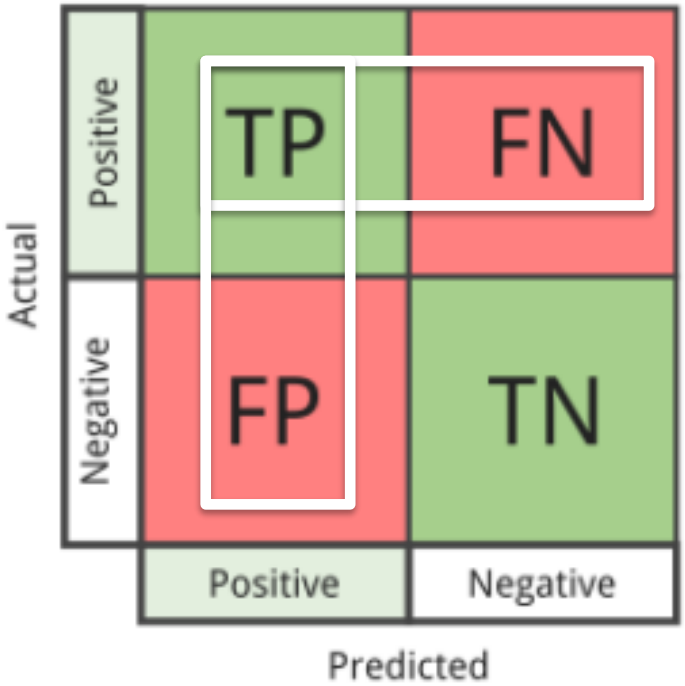
$$\text{Prevalence} = \frac{12 \text{ total dogs}}{22 \text{ total images}}$$

$$\text{Accuracy} = \frac{5 \text{ true pos.} + 7 \text{ true neg.}}{22 \text{ total images}}$$

hogeschool vives

Recall and Precision: Confusion Matrix

- **Recall =** $\frac{TP}{TP+FN}$
- **Precision =** $\frac{TP}{TP+FP}$



RECALL

PRECISION

Recall: Example

- Simple test:
  - Accuracy: 99%
  - Recall: 0 / (0 + 10) = 0

Coronavirus model 1	Predicted sick	Predicted healthy	Total
Actual sick	0	10	10
Actual healthy	0	990	990
Total	0	1000	1000

- Better test:
  - Accuracy: 95%
  - Recall: 8 / (8 + 2) = 0.8
  - Less FN!

Coronavirus model 2	Predicted sick	Predicted healthy	Total
Actual sick	8	2	10
Actual healthy	48	942	990
Total	56	944	1000

- Solution: let everyone test positive?
  - Recall = 100%
  - BUT: too much FP → precision = 1%!

hogeschool vives

Precision: Example

Spam filter 1	Predicted spam	Predicted ham	Total
Spam	30	10	40
Ham	5	55	60
Total	35	65	100

Spam filter 2	Predicted spam	Predicted ham	Total
Spam	35	5	40
Ham	10	50	60
Total	45	55	100

$$\text{accuracy}_1 = \frac{85}{100} = 0.85 = 85\%$$

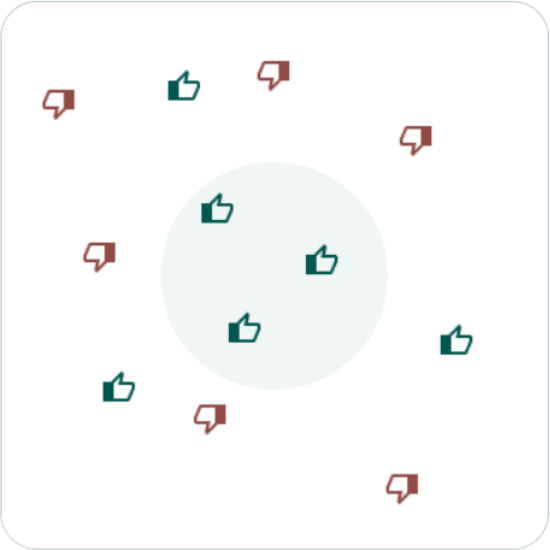
$$\text{accuracy}_2 = \frac{85}{100} = 0.85 = 85\%$$

$$\text{precision}_1 = \frac{30}{30+5} = 85.7\%$$

$$\text{precision}_2 = \frac{35}{35+10} = 77.7\%$$

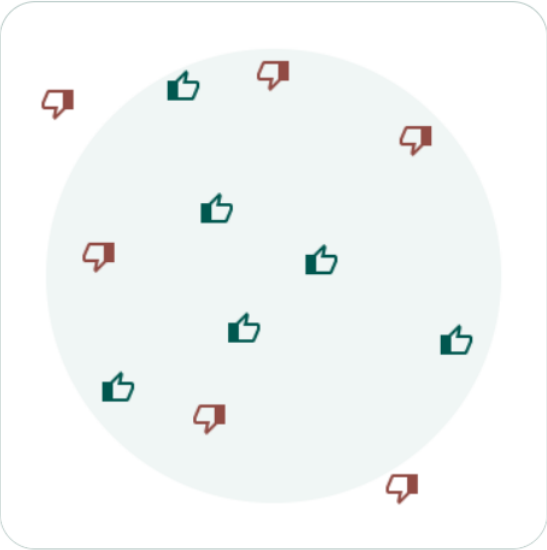
- First spam filter has greater precision!
  - What we want is: keep FP as low as possible
- Solution: Send all emails to inbox?
  - Precision = 100% (= 0/0)
  - BUT: too much FN → recall = 0%!

Recall and Precision: Practical



Precision

No false positives are classified, but some true positives are missed.



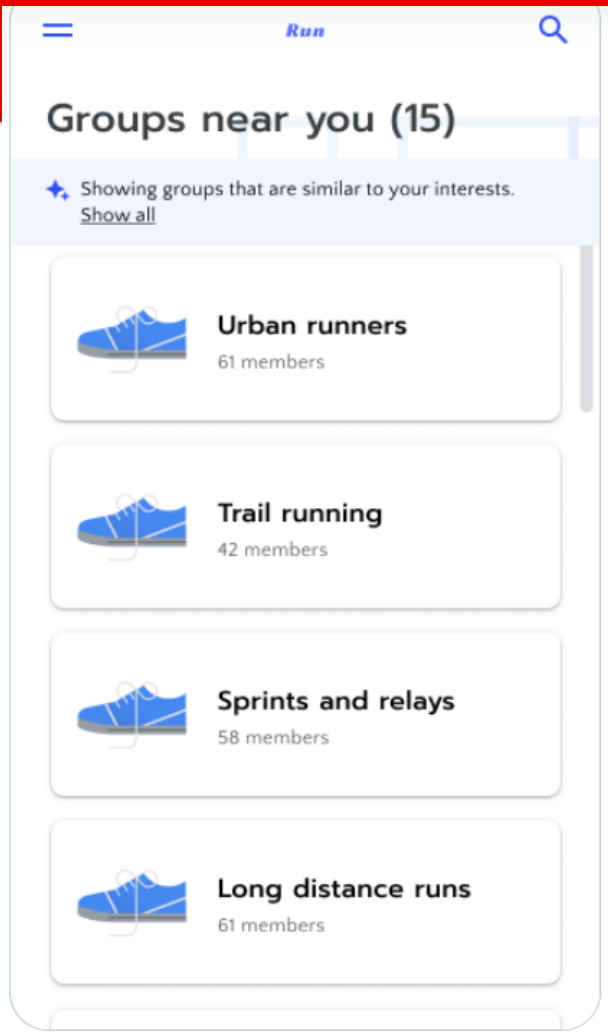
Recall

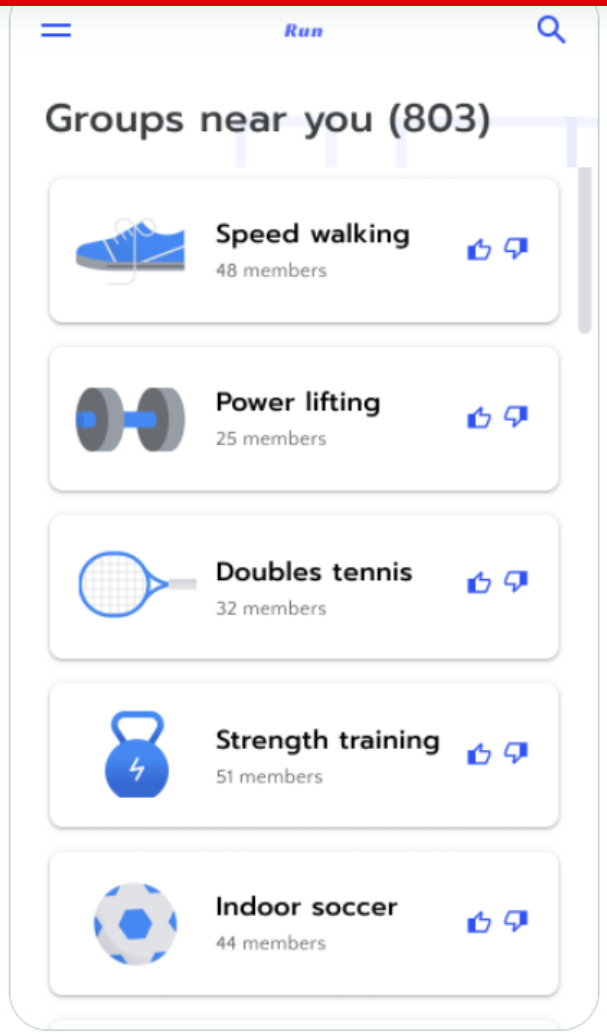
All true positives are classified, but some false positives are captured.

https://pair.withgoogle.com/guidebook/patterns

- **Prioritize precision:** If you are building a product in a high stakes domain (such as healthcare), and the risks stemming from an error are high, you will likely want to be more conservative with your model output, and only give recommendations for the things the system is most confident of. This means prioritizing precision.
- **Prioritize recall:** if your product is in a lower stakes domain and it's not a problem for users to sort through a longer list of recommendations (for example, music recommendations), you might prefer to prioritize recall. This means your user will get a larger set of results, offering the chance for surprising or serendipitous discoveries, but some of them may be less relevant.

Recall and Precision: Example





Precision

Enable users to include results (true positives) that may have been excluded.

Recall

Enable users to exclude results (false positives) that may have been included.

F_1-score: Definition

- Conclusion: we need to look at recall and precision!
- ***F_1*-score**: metric that combines **recall and precision**
  - Arithmetic mean? No
  - **Harmonic mean**? Yes
  - Why? Recall and precision are ratios!

$$\frac{1}{F_1} = \frac{1}{2}\left(\frac{1}{P} + \frac{1}{R}\right)$$

which implies that:

$$\boxed{F_1 = \frac{2PR}{P + R}}$$

Intermezzo: Harmonic Mean

- Distance Kortrijk – Bruges = 50 km
- From Kortrijk to Bruges: 30 minutes = 100 km/h
- From Bruges to Kortrijk: 60 minutes = 50 km/h

- What is the average speed?
  - 100 km in 90 minutes = 66.66 km/h
  - Arithmetic mean = (100+50)/2 = 75 km/h ✗
  - Harmonic mean = (2x50x100)/(100+50) = 66.66 km/h ✓

- Why harmonic mean?

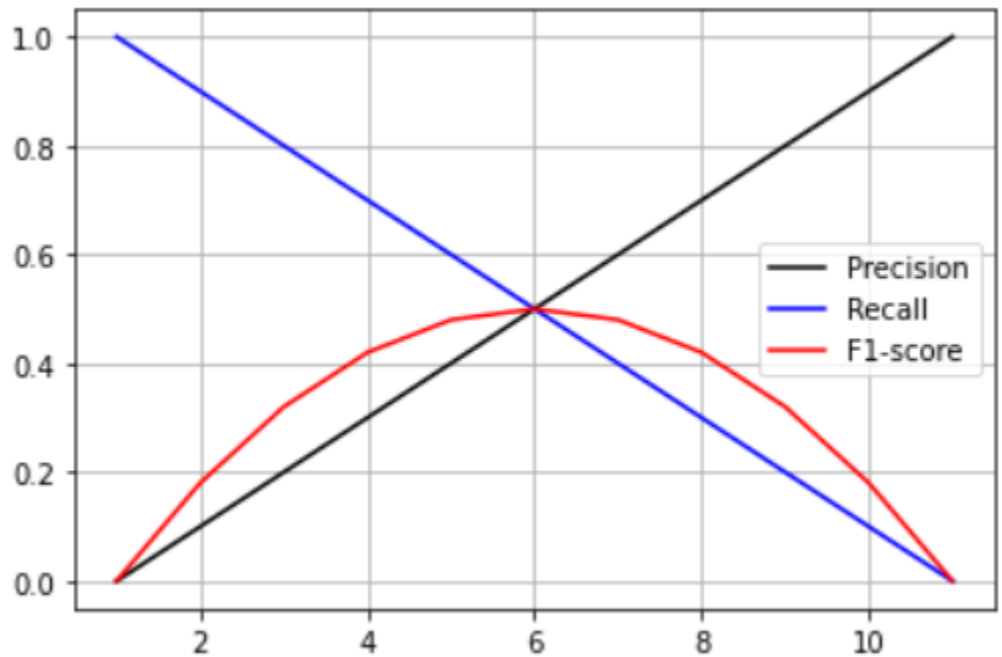
  Speeds are also ratios!

hogeschool vives

F_1-score: Interpretation

- F_1 expresses the balance between precision and recall
- If P << R or R << P, then F_1 small
- If P = R, then F_1 = P = R
- We want high F_1, because then both P and R are high!
- Example:

```
import numpy as np
import matplotlib.pyplot as plt

P = np.linspace(0, 1, 11)
R = P[-1::-1]
F1 = 2 * P * R / (P + R)
i = np.arange(1, 12)
plt.plot(i, P, 'k-', i, R, 'b-', i, F1, 'r-');
plt.legend(("Precision", 'Recall', "F1-score"))
plt.grid();
```



F_β-score: Definition

- F_1 treats precision and recall equally
- Sometimes we want to emphasize precision or recall more
- The F_β-score solves this problem!

$$\boxed{F_\beta = \frac{(1+\beta^2)PR}{\beta^2 P + R}}$$

- Examples:

$$F_{0.1} = \frac{(1+0.1^2)PR}{0.1^2P+R} = \frac{1.01PR}{0.01P+R} \qquad F_1 = \frac{(1+1^2)PR}{1^2P+R} = \frac{2PR}{P+R} \qquad F_{10} = \frac{(1+10^2)PR}{10^2P+R} = \frac{101PR}{100P+R}$$

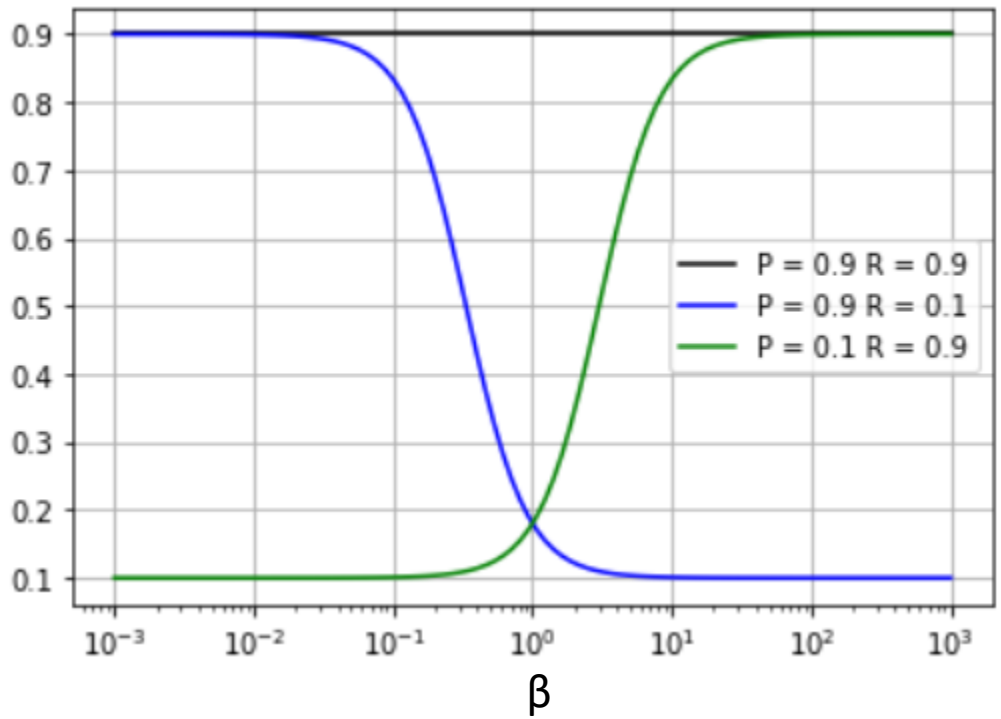
hogeschool vives

F_β-score: Interpretation

- $\beta < 1$: emphasis on P
- $\beta > 1$: emphasis on R
- If P = R, then F_β = P = R

- Example:

```
beta = np.logspace(-3, 3, 100)
beta2 = beta**2
F_beta = lambda P, R: (1 + beta2) * P * R / (beta2 * P + R)

P1, R1 = 0.9, 0.9
P2, R2 = 0.9, 0.1
P3, R3 = 0.1, 0.9

plt.semilogx(beta, F_beta(P1, R1), "k-",
             beta, F_beta(P2, R2), "b-",
             beta, F_beta(P3, R3), "g-");
plt.legend(["P = 0.9 R = 0.9",
            "P = 0.9 R = 0.1",
            "P = 0.1 R = 0.9"])
plt.grid();
```



Sensitivity and Specificity: Definitions

- **Sensitivity** $= \frac{TP}{TP+FN}$

  = percentage of correctly classified positive samples
  = Recall!
  = Positive recall
  = True positive rate (TPR)

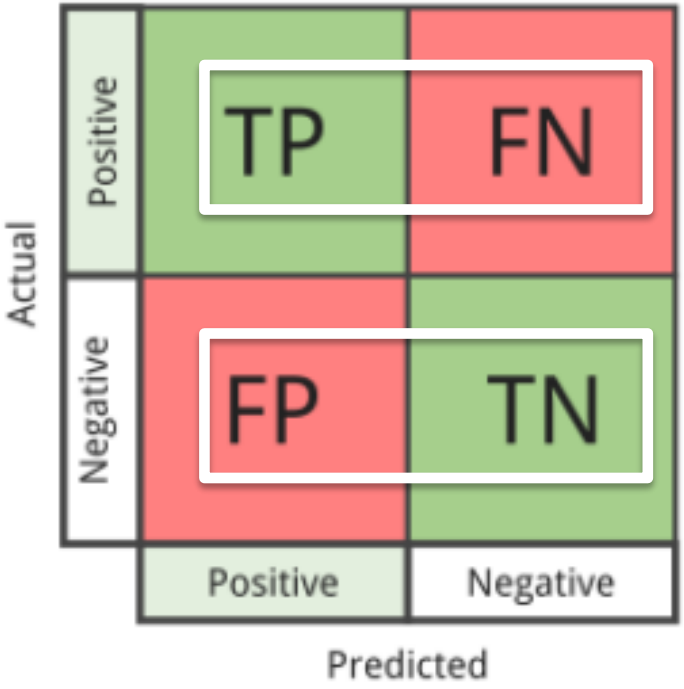
- **Specificity** $= \frac{TN}{TN+FP}$

  = percentage of correctly classified negative samples
  = Negative recall
  = True negative rate (TNR)

hogeschool vives

Sensitivity and Specificity: Confusion Matrix

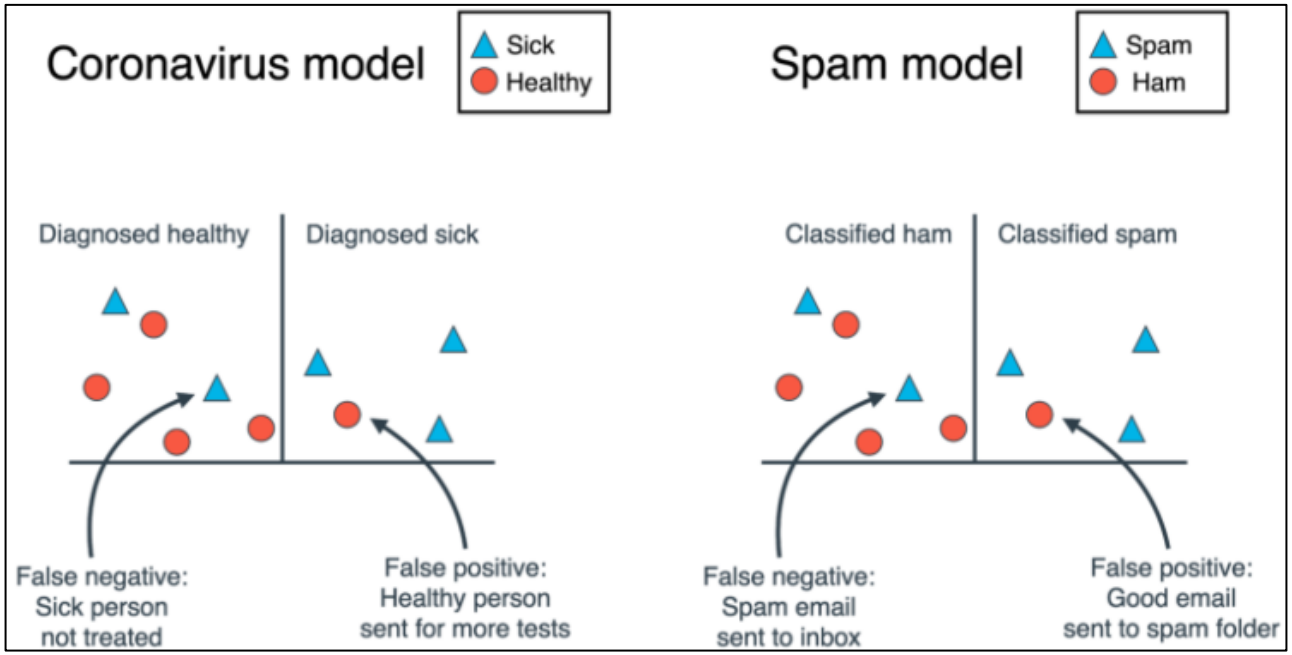
- **Sensitivity =** $\frac{TP}{TP+FN}$
- **Specificity =** $\frac{TN}{TN+FP}$



SENSITIVITY = RECALL

SPECIFICITY

Sensitivity and Specificity: Example



- 3 true positives
- 4 true negatives
- 1 false positive
- 2 false negatives

$$\text{sensitivity} = \frac{\text{true positives}}{\text{true positives}+\text{false negatives}} = \frac{3}{3+2} = 0.6 = 60\%$$

$$\text{specificity} = \frac{\text{true negatives}}{\text{true negatives}+\text{false positives}} = \frac{4}{4+1} = 0.8 = 80\%$$

False Positive and False Negative Rate: Definitions

- **Sensitivity** $= \frac{TP}{TP+FN}$
  = True positive rate (TPR)

- **Specificity** $= \frac{TN}{TN+FP}$
  = True negative rate (TNR)

- **False positive rate** $= \frac{FP}{FP+TN}$ = 1 - TNR

- **False negative rate** $= \frac{FN}{FN+TP}$ = 1 - TPR

Relationship between Metrics: Confusion Matrix

	Predicted positive	Predicted negative
Positive label	True positives	False negatives
Negative label	False positives	True negatives

sensitivity (recall, true positive rate) = TP / (TP + FN)

specificity = TN / (TN + FP)

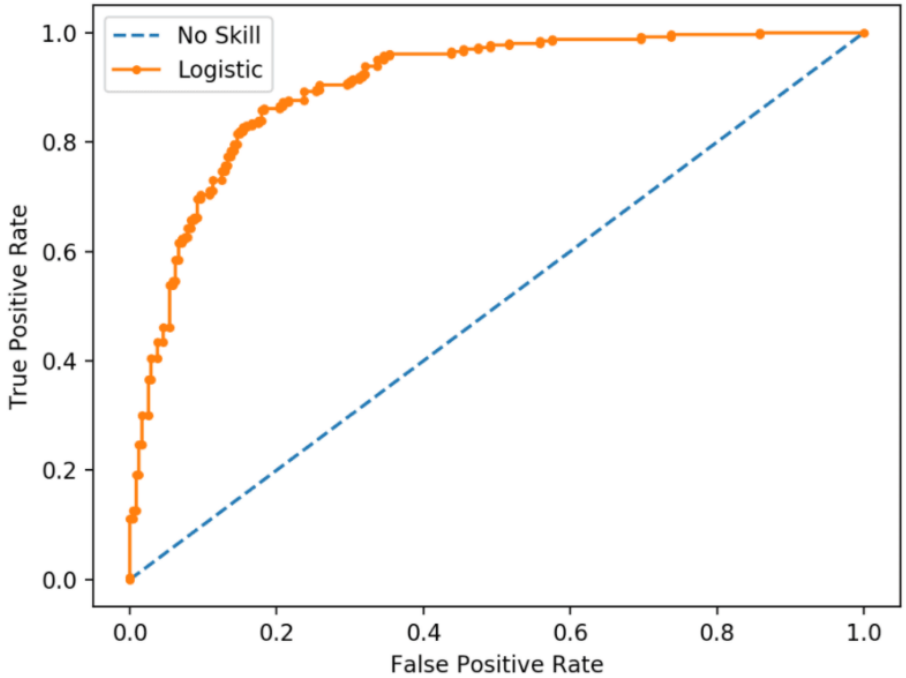
false positive rate = FP / (FP + TN)

precision = TP / (TP + FP)

hogeschool vives

ROC Curve

- ROC = Receiver Operating Characteristic
- Comes from signal detection theory
- TPR vs FPR = sensitivity vs (1 – specificity)
- Finding the best threshold value for a binary classifier



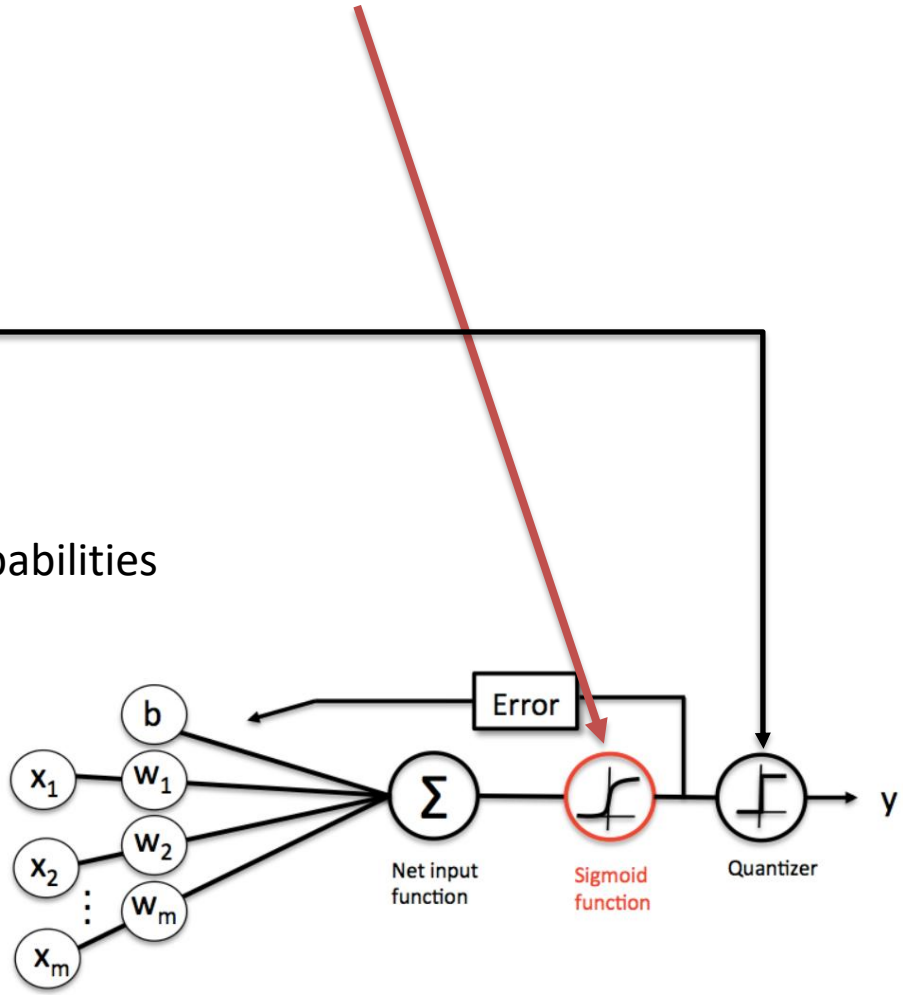
Logistic Regression

1. Transforming weighted sum into probability (sigmoid function)

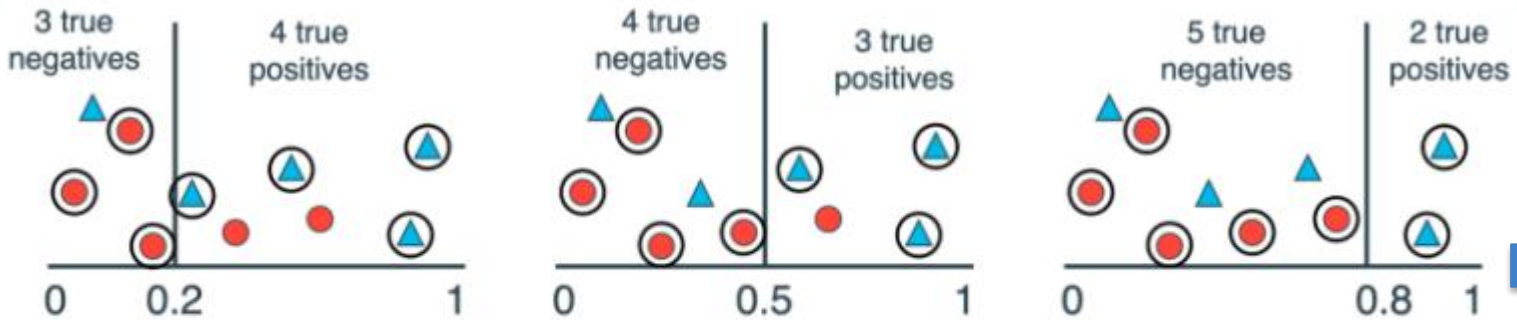
```
def predict(w, b, X):
  return sigmoid(np.dot(X, w) + b)

predict(w, b, X)
```

2. Convert probabilities to 0 or 1
   - Select threshold (by user!):
     - probabilities > threshold → 1
     - probabilities < threshold → 0
   - E.g. threshold = 0.5: round off probabilities

```
# labels
np.round(predict(w, b, X))
```



Specificity and Sensitivity for Different Thresholds: Example



Threshold = 0.2
Sensitivity = 4/5
Specificity = 3/5

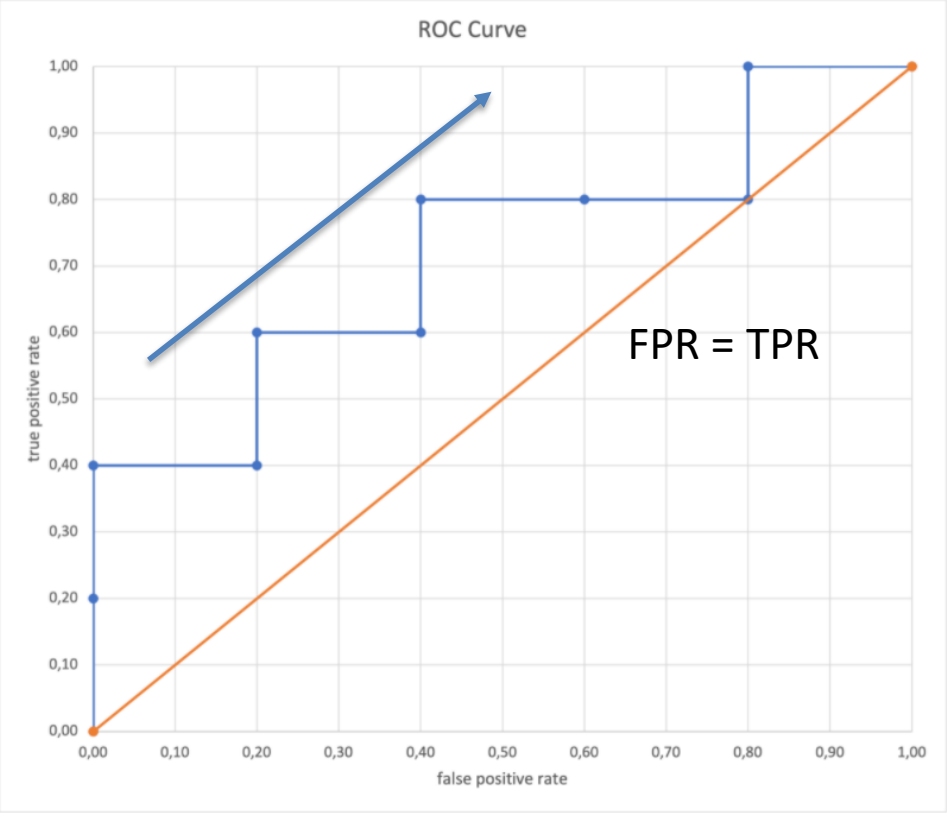
Threshold = 0.5
Sensitivity = 3/5
Specificity = 4/5

Threshold = 0.8
Sensitivity = 2/5
Specificity = 5/5

- The lower the threshold, the higher the sensitivity
- The higher the threshold, the higher the specificity
- The higher the threshold, the lower the false positive rate

	Threshold 0.2	Threshold 0.5	Threshold 0.8
TP	4	3	2
TN	3	4	5
FP	2	1	0
FN	1	2	3
Sensitivity	80%	60%	40%
Specificity	60%	80%	100%
False positive rate	40%	20%	0%

Creating a ROC Curve



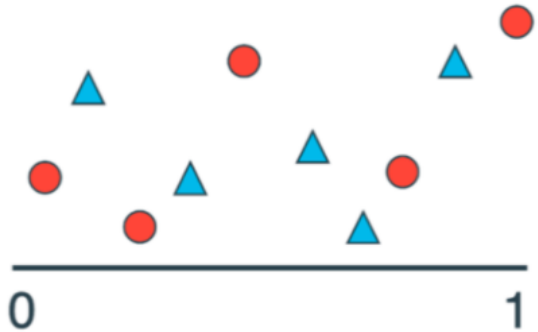
- Diagonal = random classifier
  = “guessing” = 50% accuracy

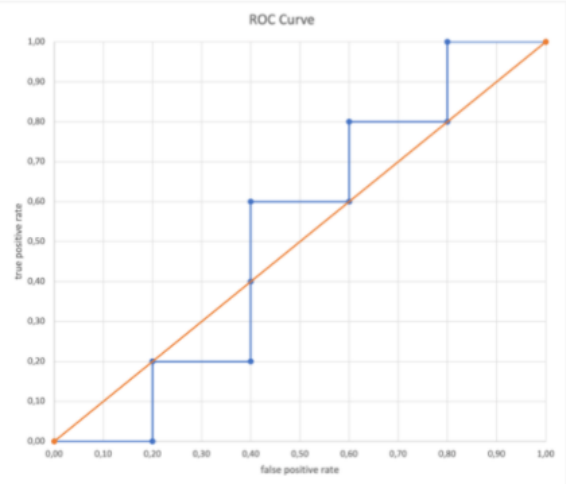
- Take point furthest from diagonal
  - Here: 3 points
  - Choice depends on application

threshold	true positives	false negatives	true negatives	false positives	controle	sensitivity	specificity	false positive rate	accuracy
0	5	0	0	5	10	1	0	1	0,5
0,1	5	0	1	4	10	1	0,2	0,8	0,6
0,2	4	1	1	4	10	0,8	0,2	0,8	0,5
0,3	4	1	2	3	10	0,8	0,4	0,6	0,6
0,4	4	1	3	2	10	0,8	0,6	0,4	0,7
0,5	3	2	3	2	10	0,6	0,6	0,4	0,6
0,6	3	2	4	1	10	0,6	0,8	0,2	0,7
0,7	2	3	4	1	10	0,4	0,8	0,2	0,6
0,8	2	3	5	0	10	0,4	1	0	0,7
0,9	1	4	5	0	10	0,2	1	0	0,6
1	0	5	5	0	10	0	1	0	0,5

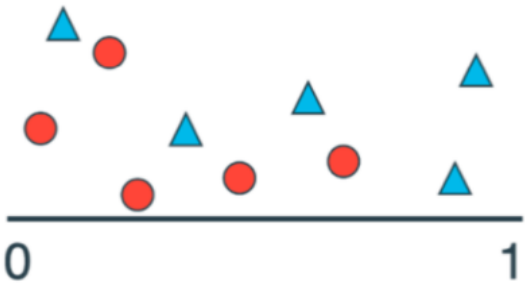
AUC = Area Under the Curve

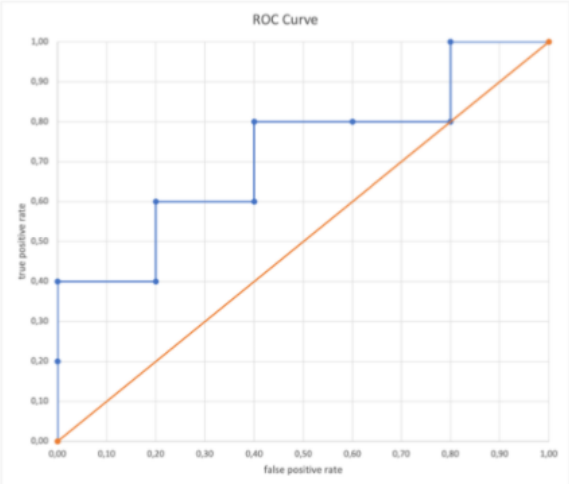
AUC = 0.52



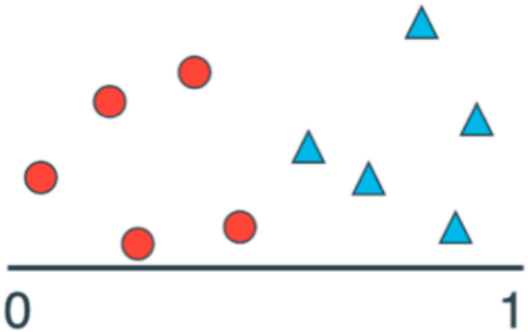


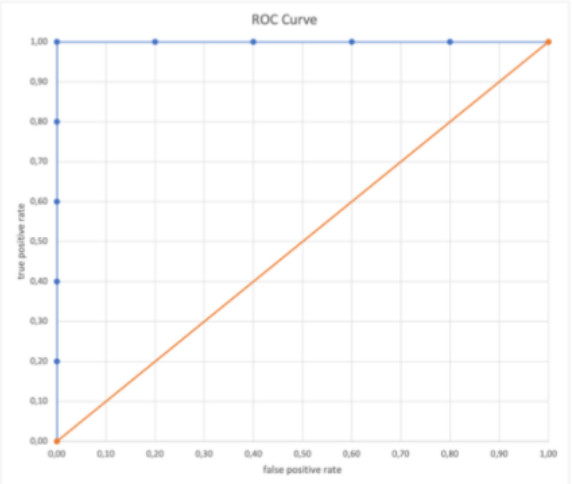
AUC = 0.72



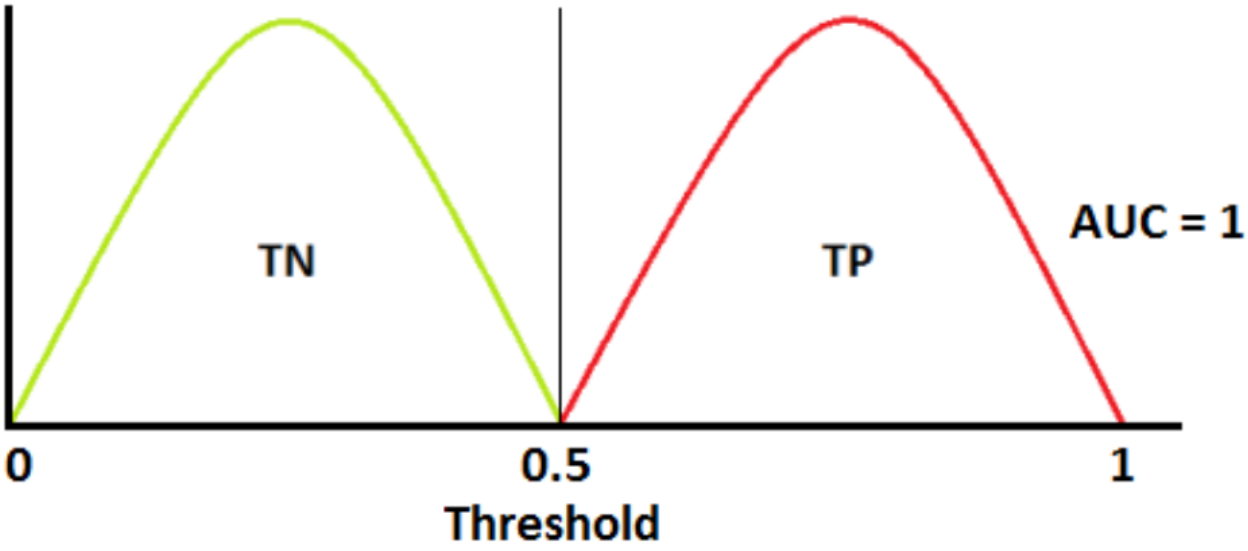


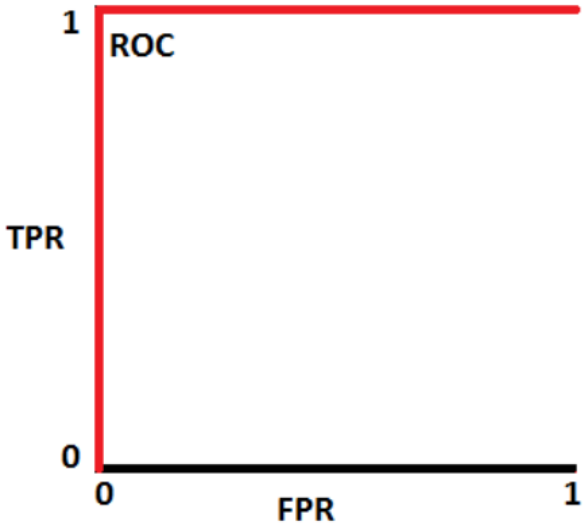
AUC = 1.00



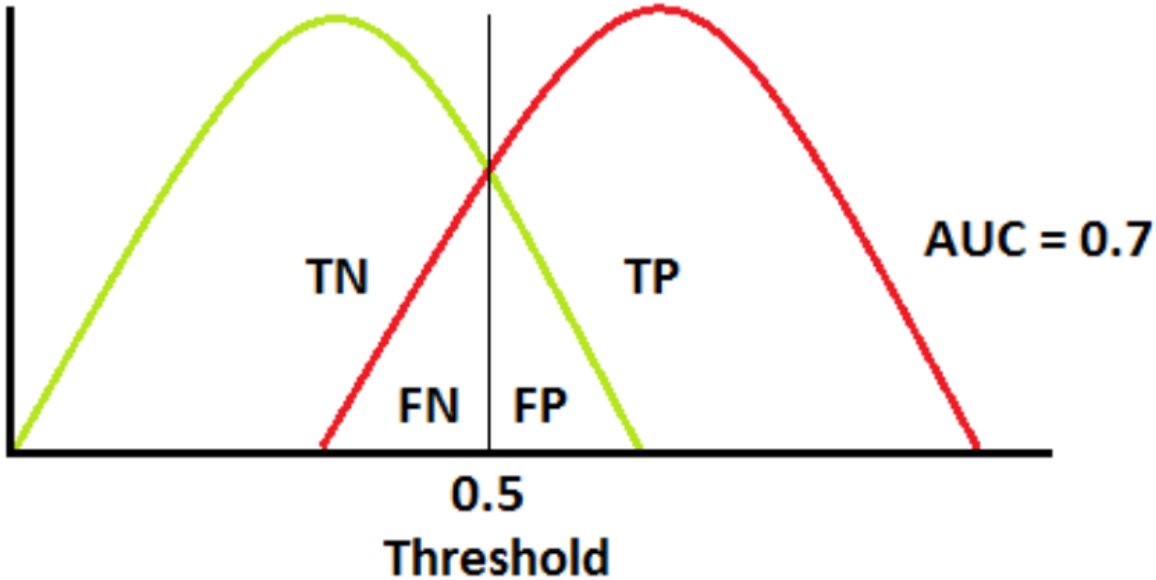


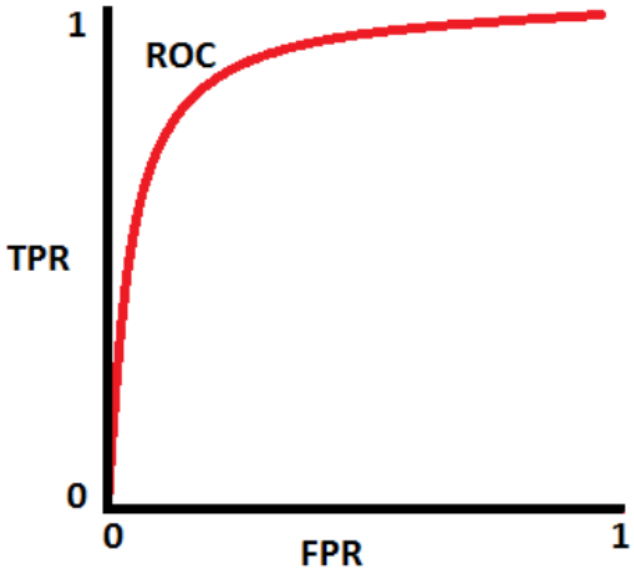
AUC: Perfect Classifier



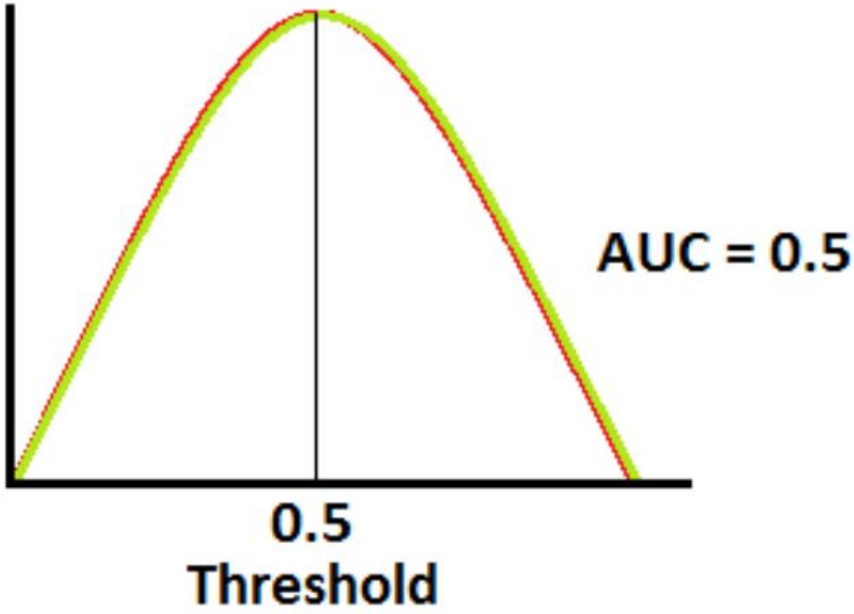


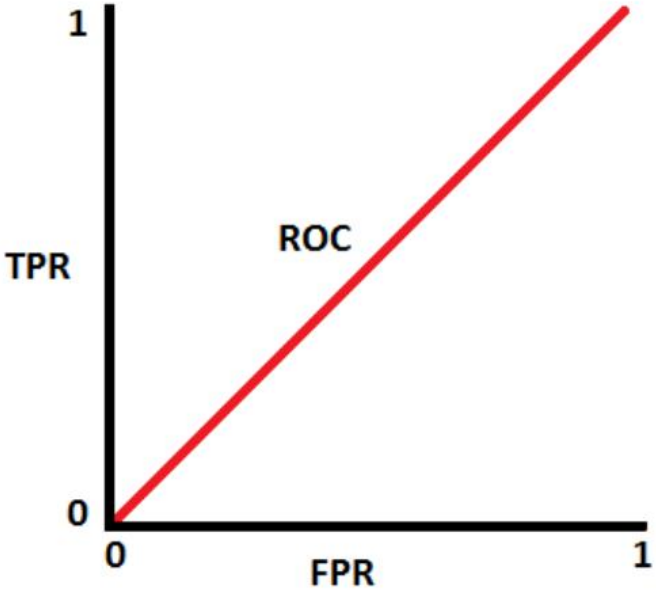
AUC: Good Classifier



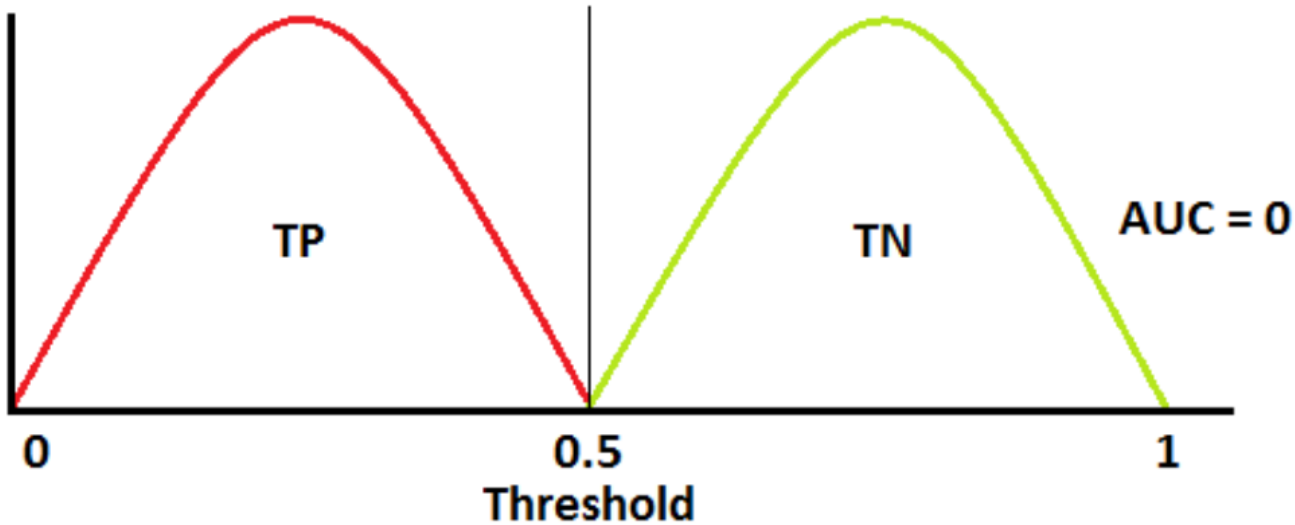


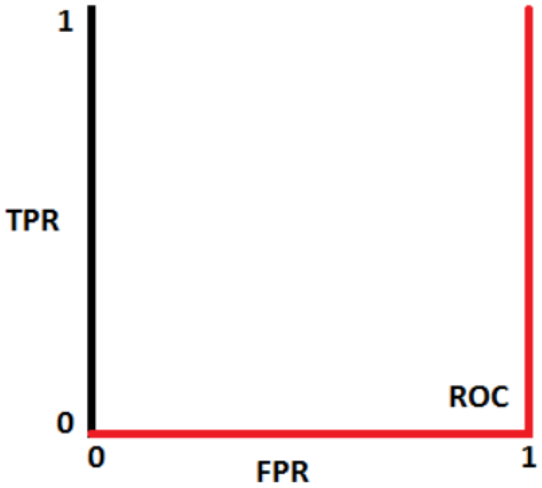
AUC: Random Classifier



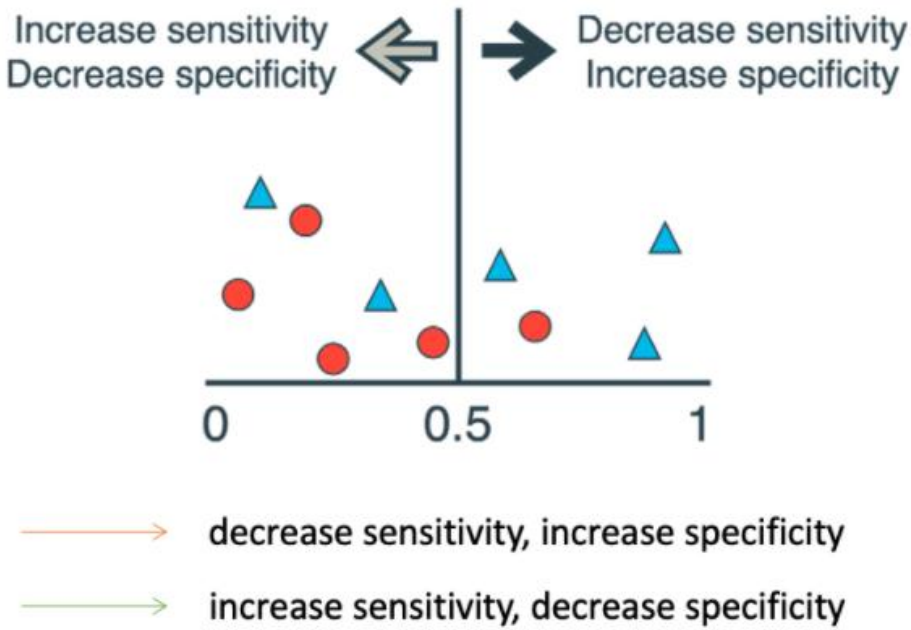


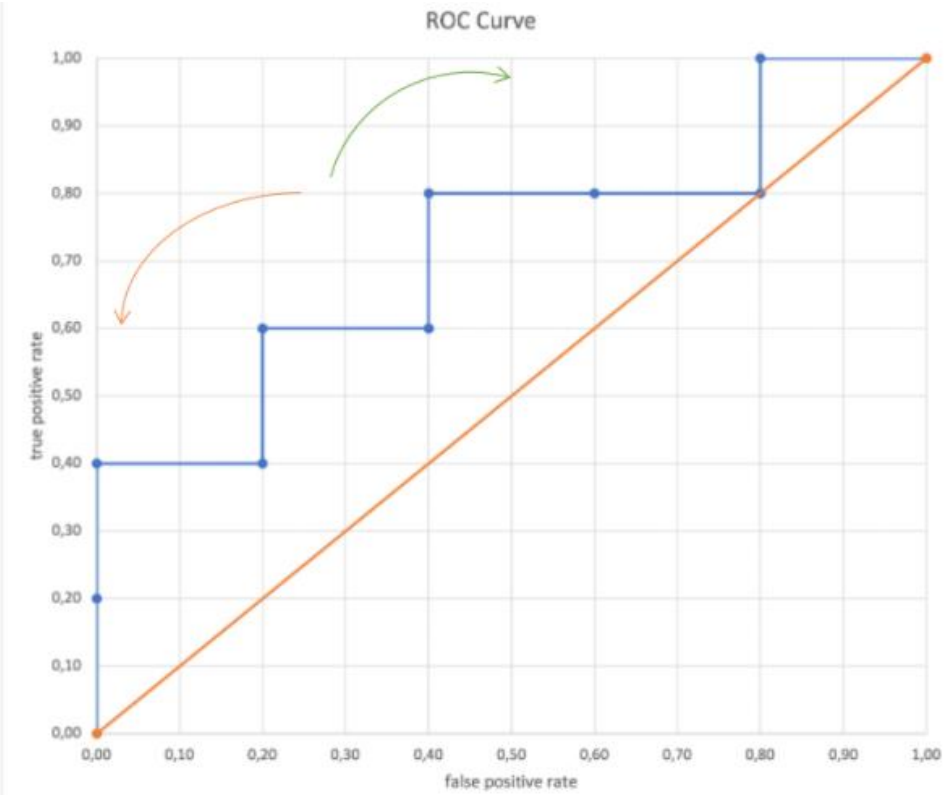
AUC: Inverse Classifier





Select Threshold based on ROC Curve



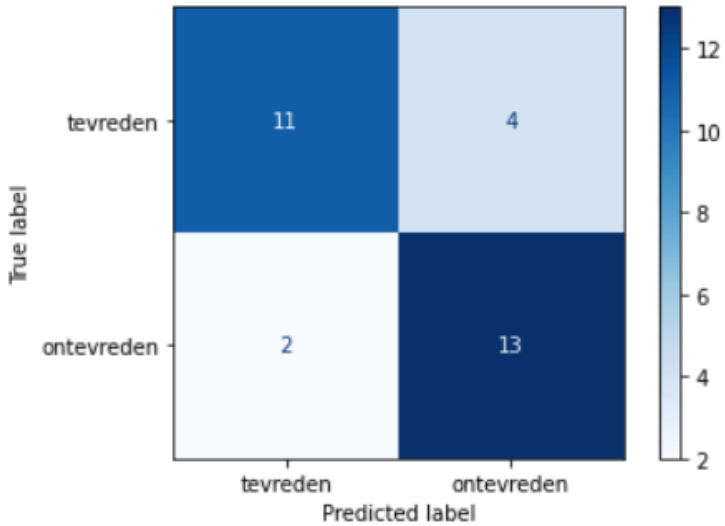


hogeschool vives

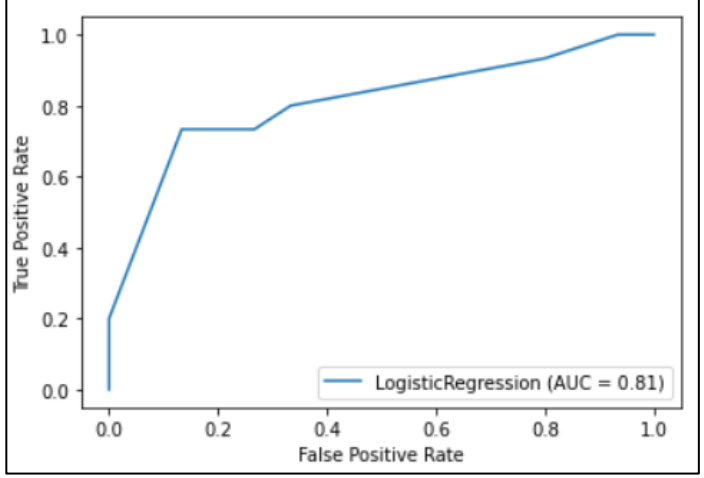
Scikit-Learn: Module metrics

```
from sklearn.metrics import plot_confusion_matrix, accuracy_score, precision_score, \
                            recall_score, f1_score, fbeta_score, plot_roc_curve, roc_auc_score, roc_curve, auc
```

```
disp = plot_confusion_matrix(perceptron, X, y,
                             labels=[1, 0],
                             display_labels=['tevreden', 'ontevreden'],
                             cmap=plt.cm.Blues)
```




```
plot_roc_curve(logistic, X, y);
```




```
accuracy_score(y, yp)
```

```
0.8
```

```
precision_score(y, yp)
```

```
0.8461538461538461
```

```
recall_score(y, yp)
```

```
0.7333333333333333
```

```
f1_score(y, yp)
```

```
0.7857142857142856
```

```
fbeta_score(y, yp, beta=1)
```

```
0.7857142857142856
```

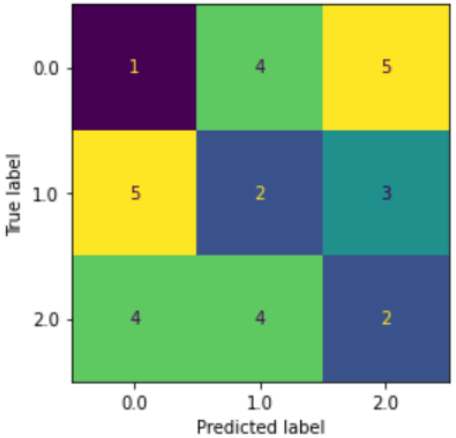
Scikit-Learn: Classification Report

The `classification_report` function builds a text report showing the main classification metrics. Here is a small example with custom `target_names` and inferred labels:

```
>>> from sklearn.metrics import classification_report
>>> y_true = [0, 1, 2, 2, 0]
>>> y_pred = [0, 0, 2, 1, 0]
>>> target_names = ['class 0', 'class 1', 'class 2']
>>> print(classification_report(y_true, y_pred, target_names=target_names))
              precision    recall  f1-score   support

     class 0       0.67      1.00      0.80         2
     class 1       0.00      0.00      0.00         1
     class 2       1.00      0.50      0.67         2

    accuracy                           0.60         5
   macro avg       0.56      0.50      0.49         5
weighted avg       0.67      0.60      0.59         5
```

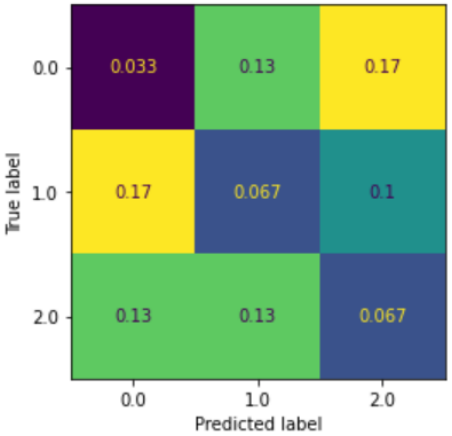
Scikit-Learn: Confusion Matrix

```
ConfusionMatrixDisplay.from_predictions(y_true=y, y_pred=yp, colorbar=False);
```



sklearn.metrics.ConfusionMatrixDisplay

```
ConfusionMatrixDisplay.from_predictions(y_true=y, y_pred=yp, normalize='all', colorbar=False);
```



Sources

- Serrano, L. G., 2020. Grokking Machine Learning (Version 12), Manning Early Access Program, Manning Publications, 500 p.

- Chapter "Evaluation Metrics" of the Data Science II Course, Professional Bachelor's Degree in Applied Computer Science, Karel de Grote Hogeschool, academic year 2019-2020, taught by Wouter Deketelaere.

- https://en.wikipedia.org/wiki/Precision_and_recall
- Online Scikit-Learn documentation: https://scikit-learn.org/stable/
- https://towardsdatascience.com/understanding-auc-roc-curve-68b2303cc9c5

hogeschool vives